UNISWAP V4 HOOKATHON & RESEARCH MASTER BLUEPRINT

Conditional Capital Commitments and Custom Delta Accounting for Heterogeneous v4 Hooks

Theme: Active Management & Curation | **Execution Path:** Path B (beforeSwapReturnDelta + ERC-6909 Flash Accounting)

---

1. Executive Summary & Core Vision

The protocol is a **Shared Liquidity Backplane and Bounded Curator Commitment Engine for Uniswap v4**. It solves the multi-pool liquidity fragmentation and curator accountability problem by allowing heterogeneous, independently authored v4 hooks and professional curators to draw deterministic, bounded backing from a common ERC-6909 reserve without surrendering their custom pricing curves or LP exit guarantees.

By utilizing Uniswap v4's **Custom Accounting (`beforeSwapReturnDelta: true`)**, an $O(1)$ ring-buffer admission controller, and EIP-712 signed curator intents, the protocol maximizes capital utilization (+70% expected volume boost) while eliminating the race-condition contention reverts ("phantom depth") and front-runnable keeper overhead that plague existing virtual-balance and active liquidity management (ALM) systems.

---

2. Problem Statement & UHI 11 Alignment

2.1 The Five Open Curation Problems & Core DeFi Bottlenecks

- **External Wrapper Overhead & Keeper Front-Running:** Active managers (ALMs) wrap Uniswap from the outside, adding trust assumptions and gas overhead. Traditional rebalances are keeper-driven and predictable, making them easy to front-run and costly to run.

- **The v4 One-Hook Silo:** Uniswap v4 attaches each pool to exactly one hook. Specialized hooks (LVR dynamic fee hooks, RWA compliance hooks, collateralized debt hooks) must bootstrap isolated liquidity from scratch and cannot natively share LP capital across pools.
- **The "Phantom Depth" Contention Hazard in Shared Reserves:** Existing shared-balance protocols (1inch Aqua, EulerSwap, Fluid) rely on purely atomic execution-time balance checks. When a single USDC reserve backs multiple markets, all markets advertise 100% of that depth to off-chain aggregators and solvers. During volatility spikes, concurrent multi-hop routes hit multiple legs; the first swap drains the reserve, causing subsequent transactions to hard-revert.
- **Unverified Track Records & Unbounded Curator Risk:** Depositors have no on-chain way to verify a manager's track record before following them, and curator economics (performance fees, upgrades, risk limits) are bespoke per vault with no shared standard.
- **The 50% Fee Ceiling & Deadweight Loss:** Constant-product AMMs and isolated dynamic-fee hooks cap LP fee capture at ~50% of the maximum dislocation value, leaving over 25% of dislocation value unrealized or leaked to external searchers.

2.2 Existing Solutions & Their Limitations

Protocol	Core Approach	Structural Limitation	How Our Protocol Solves It
1inch Aqua	Virtual wallet balances (`ship`, `dock`, `pull`, `push`) shared across strategies.	Enforces balance checks atomically at settlement. Concurrent orders targeting the same balance cause race-condition reverts ("phantom depth").	Adds an on-chain **Admission Controller & Commitment Registry** that partitions shared capital into guaranteed reserved slices (Σ Q_{j,a}) and an opportunistic atomic buffer.
EulerSwap (Euler v2 + v4)	Connects lending vaults to v4 hooks via JIT borrowing.	Borrowing power is unreserved; concurrent demand	Provides **time-bounded capacity**

Protocol	Core Approach	Structural Limitation	How Our Protocol Solves It
		causes severe slippage or reverts. Monolithic hook per LP.	**reservations** and curator floor allocations across multiple distinct hooks, guaranteeing committed liquidity cannot be drained by a sibling pool.
Uniswap DualPool	Integrates JIT liquidity, ERC-4626 vault utilization, and weighted allocations.	Operates as a specific isolated pool/vault architecture rather than a universal cross-hook coordination backplane.	Standardizes a lightweight `ICommitmentConsumer` adapter mixin that any independent v4 hook can inherit to draw from the shared ERC-6909 reserve.
Traditional ALMs (Arrakis, Gamma, Bunni)	External wrapper vaults or isolated single-pool tick rebalancers.	High gas overhead on `modifyLiquidity` tick shifts; isolated pool TVL; vulnerable to tick-rounding accounting exploits.	Uses **Path B (`beforeSwapReturnDelta`)** to inject bounded shared liquidity without minting/burning ticks, while integrating ERC-4626 vaults (Steakhouse / Gauntlet) as composable reserve modules.
MEV-X Homelander	Uses `afterSwap` hooks to execute internal arbitrage at	Assumes reference pool depth is unconstrained;	Underwrites deterministic cross-pool reserve

Protocol	Core Approach	Structural Limitation	How Our Protocol Solves It
	zero fee against reference pools.	degrades under finite-depth reference constraints and concurrent block contention.	capacity Q_{j,a}(t) and enables zero-fee (f = 0) internal delta reconciliation across sibling pools.

---

3. Our Proposal: The Conditional Commitment Engine (Path B Architecture)

3.1 Core Architectural Principles

- **Separation of Custody & Pricing:** `SharedCommitmentVault` custodies shared reserves as ERC-6909 claims inside the singleton `PoolManager`. Independent v4 hooks act as consumers (`ICommitmentConsumer`) via Path B custom accounting (`beforeSwapReturnDelta`).
- **The Solvency & LP Exit Invariant:**

R_a(t) + Σ_{j ∈ A(t)} Q_{j,a}(t) ≤ C_a(t)

where C_a(t) is physical ERC-6909 backing, R_a(t) is the mandatory LP withdrawal and risk buffer (guaranteeing unconditional LP exit rights), and Q_{j,a}(t) is the maximum remaining gross outflow obligation for active commitment or curator strategy j.

- **O(1) Ring-Buffer Expiry (Zero Keeper Dependency):** Time is discretized into buckets of width δ. Obligations are stored in a fixed K-slot ring buffer. Expired commitments release automatically in $O(1)$ gas at block timestamp transition without off-chain keepers.
- **Tri-Tier Liquidity Cascade:**
    a. *Tier 1 (Committed Tranche Q_{j,a}):* Guaranteed execution for EIP-712 signed curator intents, solvers, and curator floor quotas.
    b. *Tier 2 (Uncommitted Atomic Spillover C_a - R_a - Σ Q_{j,a}):* First-come, first-served shared depth capturing +70% utilization.

c. *Tier 3 (Dynamic Fee Surge):* Escalates fees via `LPFeeLibrary.OVERRIDE_FEE_FLAG` when reserves approach the LP exit buffer R_a(t).

3.2 Direct Alignment with UHI 11 "White Space" & Win Conditions

- **Signed-Intent Rebalancing (`EIP-712`):** Curators sign target capacity and pricing parameters off-chain; the hook verifies the signature and bounds inside `beforeSwap` via `hookData` and executes with zero trusted keeper.
- **Bounded Curator Permissions:** Curators can allocate or consume liquidity only within hook-enforced limits: maximum outflow Q_{j,a}(t) per epoch bucket, approved `ICommitmentConsumer` pools, and strict preservation of R_a(t).
- **On-Chain Curator Track Record & Performance-Gated Capacity:** An in-hook registry records each curator's realized fee markout, drawdown, and High-Water-Mark performance per epoch. A curator's maximum allowed commitment ceiling Q_{j,a}^{max} scales automatically as verified returns accumulate.
- **Atomic Strategy Version Migration:** Because backing resides in `SharedCommitmentVault` as ERC-6909 claims rather than isolated pool ticks, depositors migrate from strategy Hook v1 to Hook v2 atomically behind a timelock and opt-out window without withdraw-and-redeposit gas drag.
- **Real ALM / ERC-4626 Curator Integration:** Integrates an institutional ERC-4626 vault adapter (e.g., Steakhouse / Gauntlet Morpho vault) for idle unreserved capital and optional **Reactive Network** event-driven risk callbacks (`CuratorRiskControllerRSC`) for autonomous cross-pool circuit breakers and drawdown slashing.

---

4. Endogenous Revenue Model & Unit Economics

- **Commitment Premium (Mode B):** 0.5 to 3.0 bps upfront reservation fee scaled hyperbolically by vault utilization U_a(t) = (R_a + Σ Q) / C_a, doubling as anti-griefing protection:

Fee(Q_{j,a}, k) = Q_{j,a} × k × (f_base + f_slope × U_a(t) / (1 - U_a(t)))

- **JIT Shared-Liquidity Spread:** 10%–20% protocol cut of LP swap fees earned on shared JIT tranches.
- **Tier-3 Scarcity Surge Fees:** +10 to +100 bps dynamic fee surcharges during high contention.

- **High-Water-Mark Curator Performance Fee:** 5%–10% on net excess yield above the previous peak share price, settled per epoch.
- **Unit Economics ($10M Shared Reserve Benchmark):** Raises net LP APY from a 9.1% static baseline to ~17.0% net APY while generating ~$426k/year in protocol and curator revenue.

---

5. End-to-End Development Phases & Milestones

Phase 1: Mechanism Design, Formal Proofs & Simulation

- **Tasks:**
    a. Formalize Theorems 1 (Monotonic Solvency), 2 (Keeper-Free Ring-Buffer Liveness), and 3 (Hyperbolic Griefing Bound).
    b. Build Python discrete-demand simulation and historical mainnet trace replayer (ETH/USDC vs. WBTC/USDC volatility data).
    c. Generate the 3×3 Pareto Frontier matrix comparing Static Partitioning, Pure Atomic Shared, Hard Reservation, and Tri-Tier Conditional Commitments.
- **Deliverables:** Mathematical specification draft (Sections 1–4 of research paper) and simulation validation report.

Phase 2: Solidity Core Contracts & Path B Implementation

- **Tasks:**
    a. Code `SharedCommitmentVault.sol` with ERC-6909 claim custody, $O(1)$ ring-buffer expiry, Bounded Curator Permissions, and High-Water-Mark track record accounting.
    b. Code `BaseCompatibleHook.sol` implementing `beforeSwapReturnDelta`, EIP-712 signed curator intent verification, and transient state passing (`tstore`/`tload`).
    c. Code two heterogeneous hooks (`HookA_DynamicFee` and `HookB_BoundedImpact`) plus an ERC-4626 Steakhouse/Gauntlet vault adapter and optional `CuratorRiskControllerRSC.sol` Reactive module.
- **Deliverables:** Core smart contract repository with clean compilation and unit test coverage.

Phase 3: Stateful Fuzzing, Invariant Testing & Gas Optimization

- **Tasks:**
    a. Write Foundry stateful invariant suite (`invariant_solvency_bound`, `invariant_zero_keeper_expiry`, `invariant_commitment_exclusivity`, `invariant_lp_exit_rights`).
    b. Run adversarial stress tests (`test_adversarial_sandwich_and_griefing`, `test_curator_drawdown_cap`).
    c. Optimize gas consumption for Path B custom accounting and transient storage.
- **Deliverables:** 100% pass rate across stateful fuzz invariants and gas benchmark tables.

Phase 4: Unichain Mainnet Capped Deployment & SDK

- **Tasks:**
    a. Mine hook permission bits via `HookMiner` (`CREATE2`).
    b. Deploy contracts and initialize test pools on Unichain Mainnet with a $50 capped TVL safety guard.
    c. Build TypeScript/Viem SDK for curator intent signing, quota queries, and strategy registration.
- **Deliverables:** Live mainnet verification transaction hashes and developer SDK documentation.

Phase 5: Research Paper Compilation, Hook Factory UI & Submission

- **Tasks:**
    a. Compile final IEEE/Springer LaTeX manuscript and publish preprint to arXiv.
    b. Build Next.js Hook Factory & Curator Leaderboard web dashboard.
    c. Record demo video demonstrating cross-pool liquidity sharing, bounded curator quotas, and $O(1)$ bucket expiry on Unichain Mainnet.
- **Deliverables:** Final Hookathon submission package, arXiv preprint, and demo video.